



Deep Learning

Lecture 2.2: Computational Graphs



Today's Learning Objectives

Be able to answer:

- What are computational graphs?
 - Why is the abstraction useful?
- What algorithms do they enable?
 - What is a topological sort?
 - What is a forward pass?
 - What is a backward pass and how does it implement backpropagation?





Computational Graph - A directed acyclic graph (DAG) with vertices corresponding to computational primitives and edges to intermediate results of the computation.

Nodes - either variables or operations

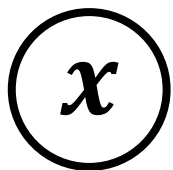
Edges - result of those computations



Expression:

x

Graph:

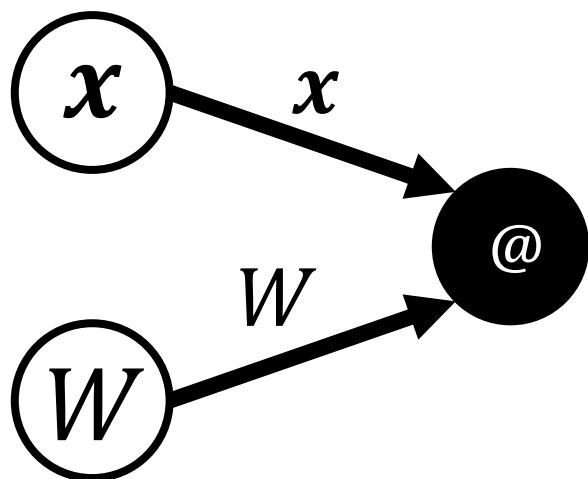




Expression:

$$\mathbf{x}W$$

Graph:

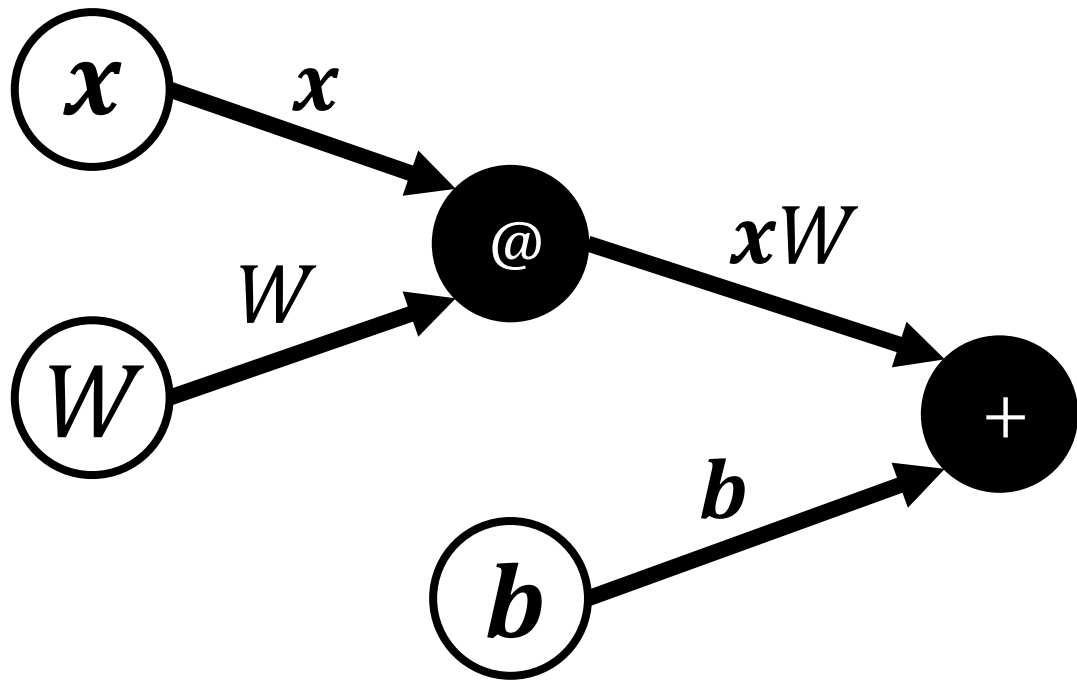




Expression:

$$\mathbf{x}W + \mathbf{b}$$

Graph:

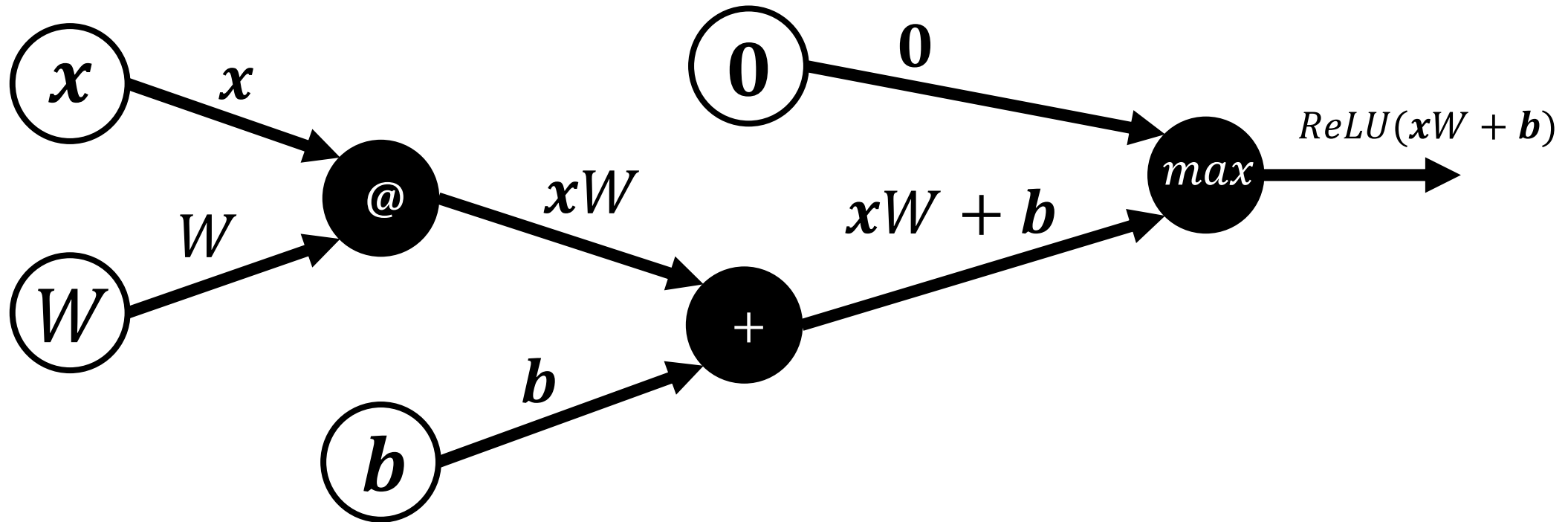




Expression:

$$\text{ReLU}(\mathbf{x}\mathbf{W} + \mathbf{b})$$

Graph:

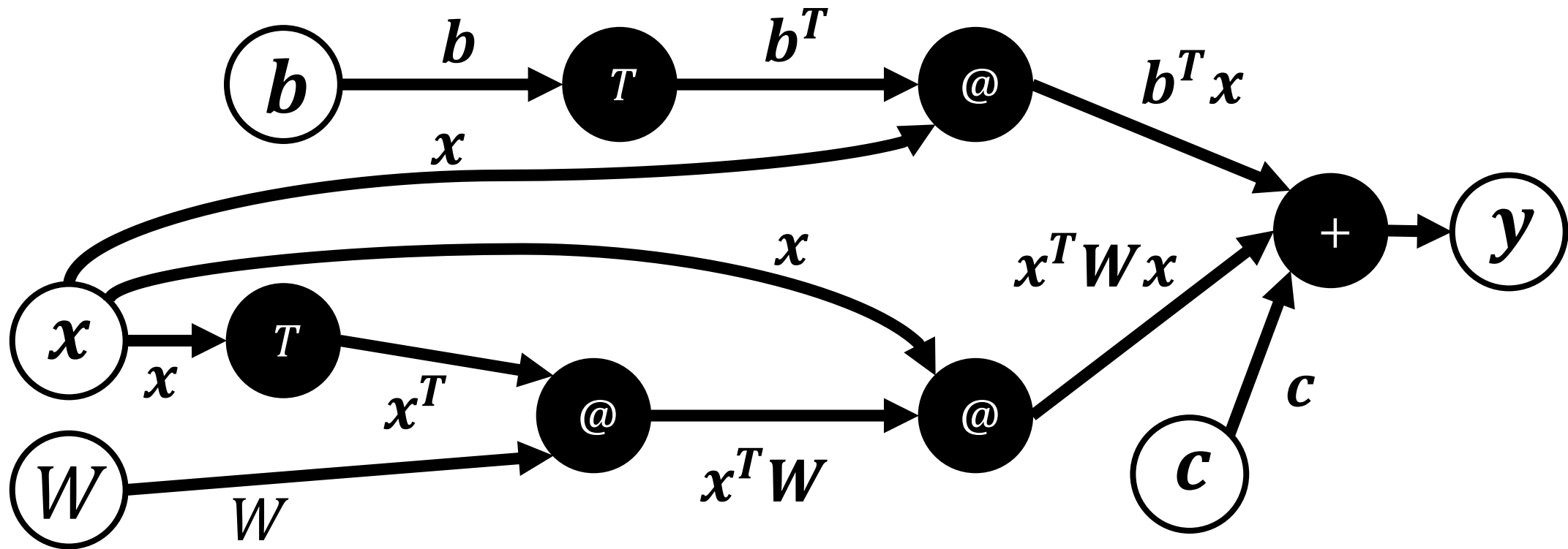




Expression:

$$y = x^T W x + b^T x + c$$

Graph:



Today's Learning Objectives

Be able to answer:

- ~~What are computational graphs?~~
- ~~Why is the abstraction useful?~~
- What algorithms do they enable?
 - What is a topological sort?
 - What is a forward pass?
 - What is a backward pass and how does it implement backpropagation?



What can we do easily with this abstraction of computational graphs?

Forward computation - compute outputs given inputs.

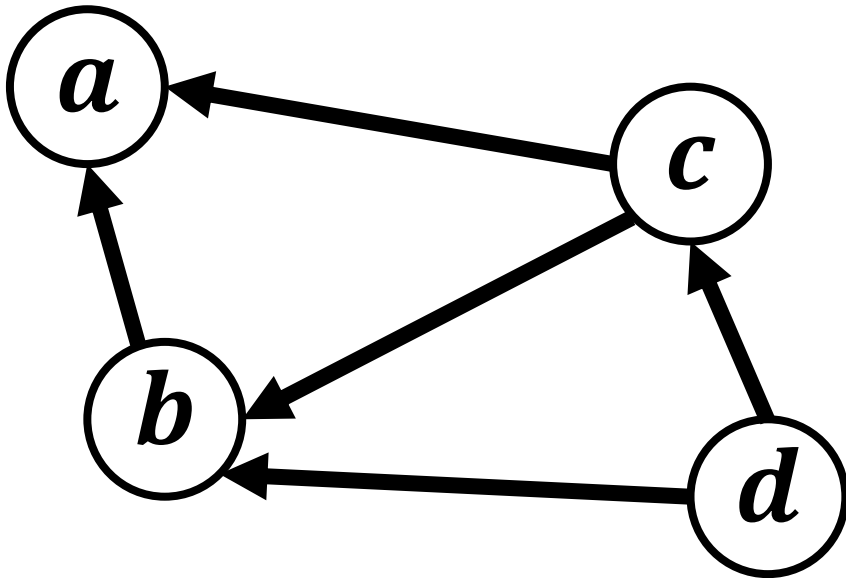
Backward computation - compute gradients.

But first.... topological ordering of graphs.

A **topological ordering** of a graph $G = (V, E)$ is an ordering of nodes in V such that v_i comes before v_j if there exists an edge (i, j) .



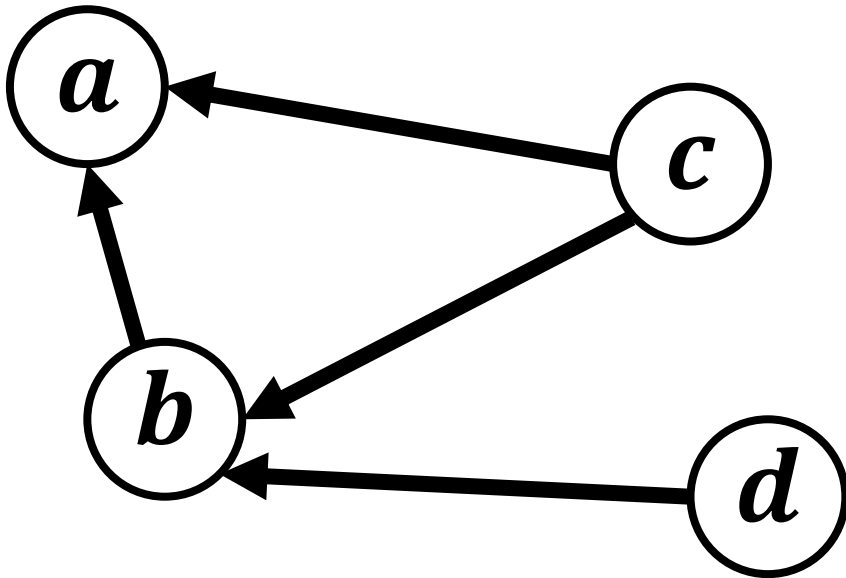
A **topological ordering** of a graph $G = (V, E)$ is an ordering of nodes in V such that v_i comes before v_j if there exists an edge (i, j) .



Example ordering:
 d, c, b, a



A **topological ordering** of a graph $G = (V, E)$ is an ordering of nodes in V such that v_i comes before v_j if there exists an edge (i, j) .



Example orderings:

d, c, b, a

c, d, b, a

What can we do easily with this abstraction of computational graphs?

Forward computation - compute outputs given inputs.

Backward computation - compute gradients.

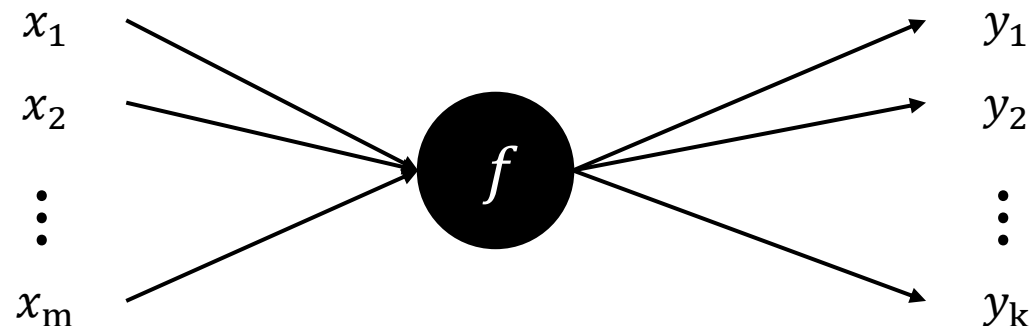
Compute outputs for a static graph

1. Perform topological ordering of graph
 - Ensures all inputs exist before performing an operation
 - Especially important if pre-allocated buffers are being overwritten
2. Execute computation according to ordering

For dynamic graph, ordering already enforced by programming language

- Can't use a variable before its declared
- Outputs generated along the way

Each node needs to encode how to make outputs from inputs:



What can we do easily with this abstraction of computational graphs?

Forward computation – compute outputs given inputs.

Backward computation – compute gradients.

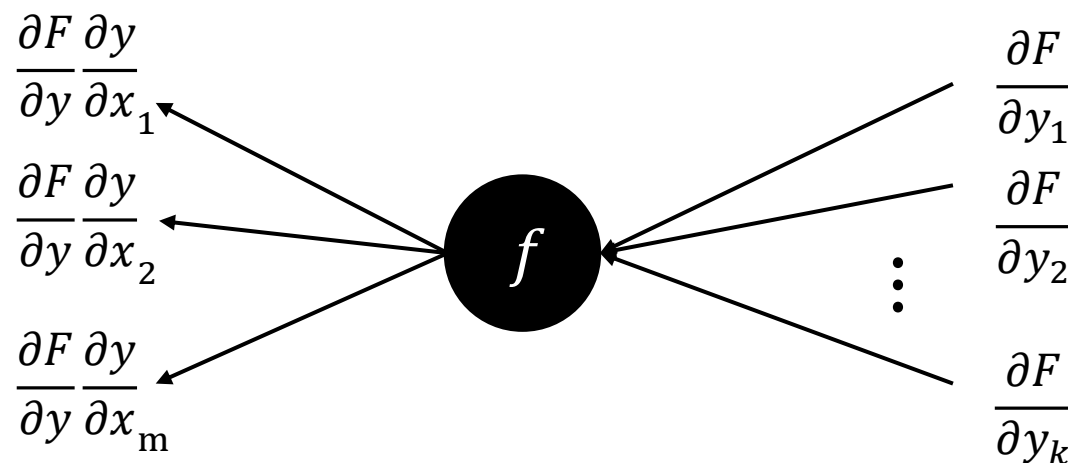


Backward Pass on Computational Graphs

Compute gradients of a node F from a graph:

1. Perform reverse topological ordering of the graph
 - Ensures all gradients are aggregated before each node passes them on
2. Execute backpropagation according to that ordering

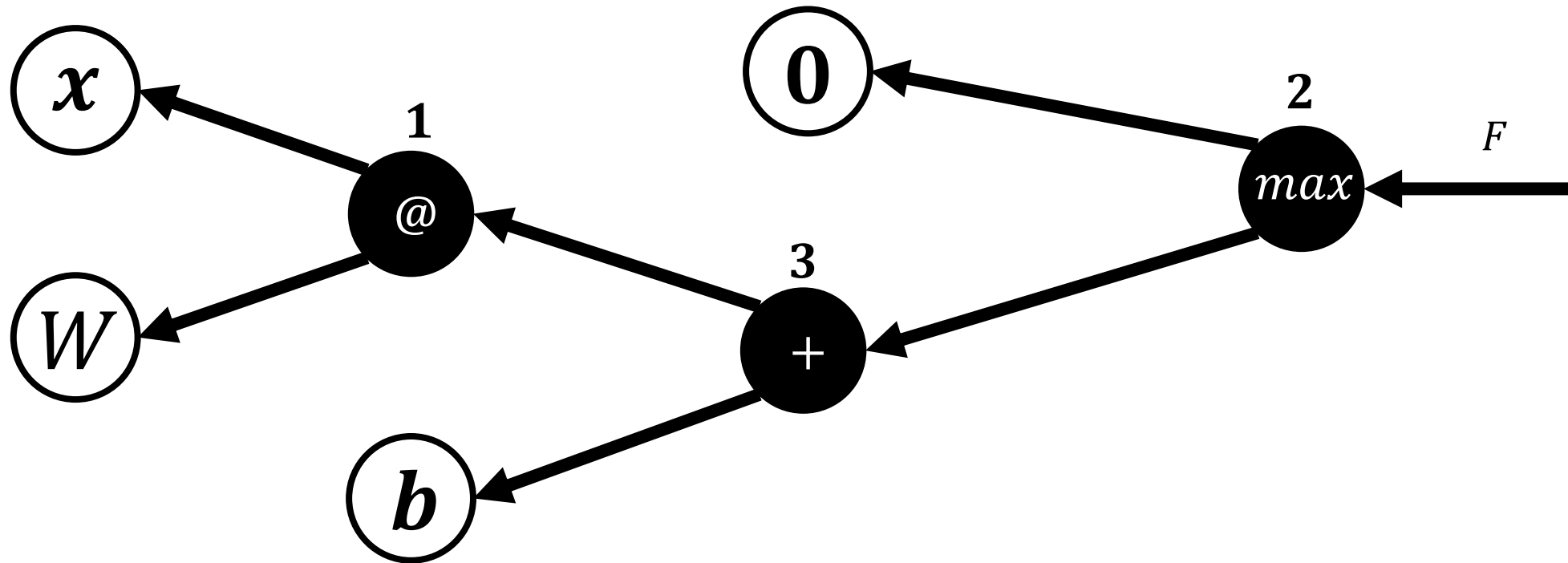
Each node needs to encode how to compute gradients of outputs y with respect to inputs x so we can compute $\frac{\partial F}{\partial y} \frac{\partial y}{\partial x}$:





A topological ordering: 2,3,1

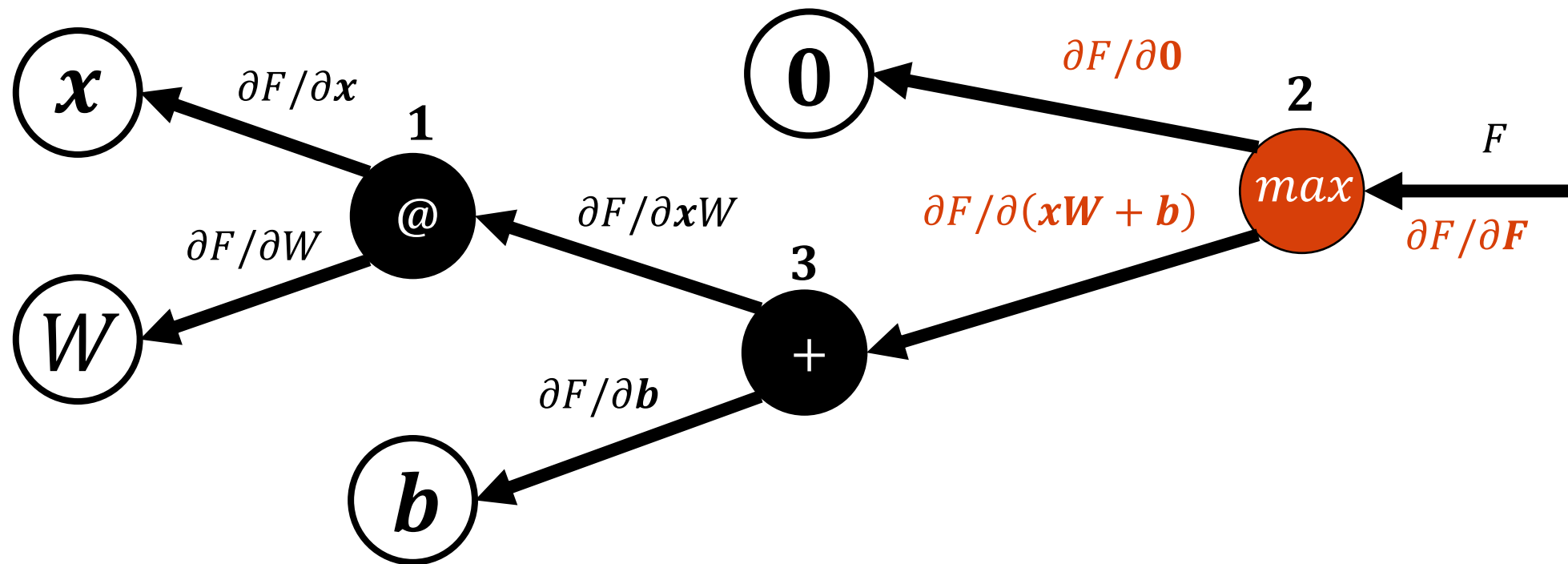
$$\frac{\partial F}{\partial \text{input}} = \frac{\partial F}{\partial \text{output}} \frac{\partial \text{output}}{\partial \text{input}}$$





A topological ordering: 2,3,1

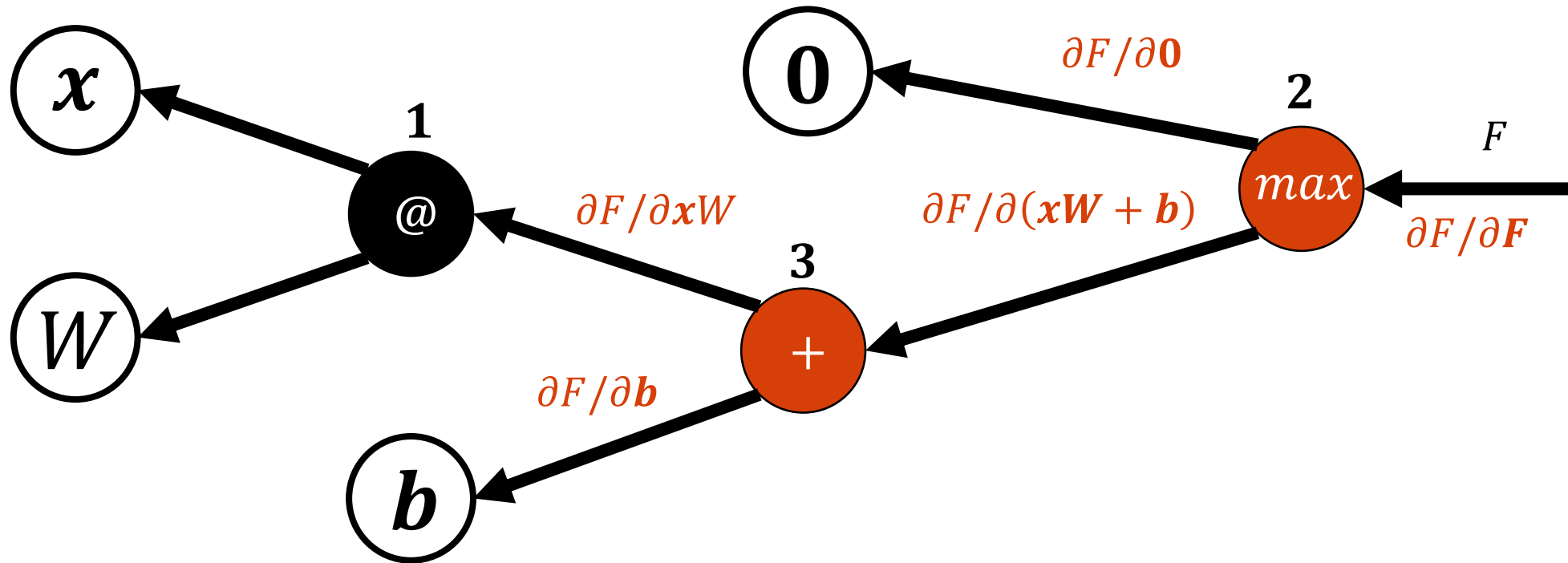
$$\frac{\partial F}{\partial \text{input}} = \frac{\partial F}{\partial \text{output}} \frac{\partial \text{output}}{\partial \text{input}}$$





A topological ordering: 2,3,1

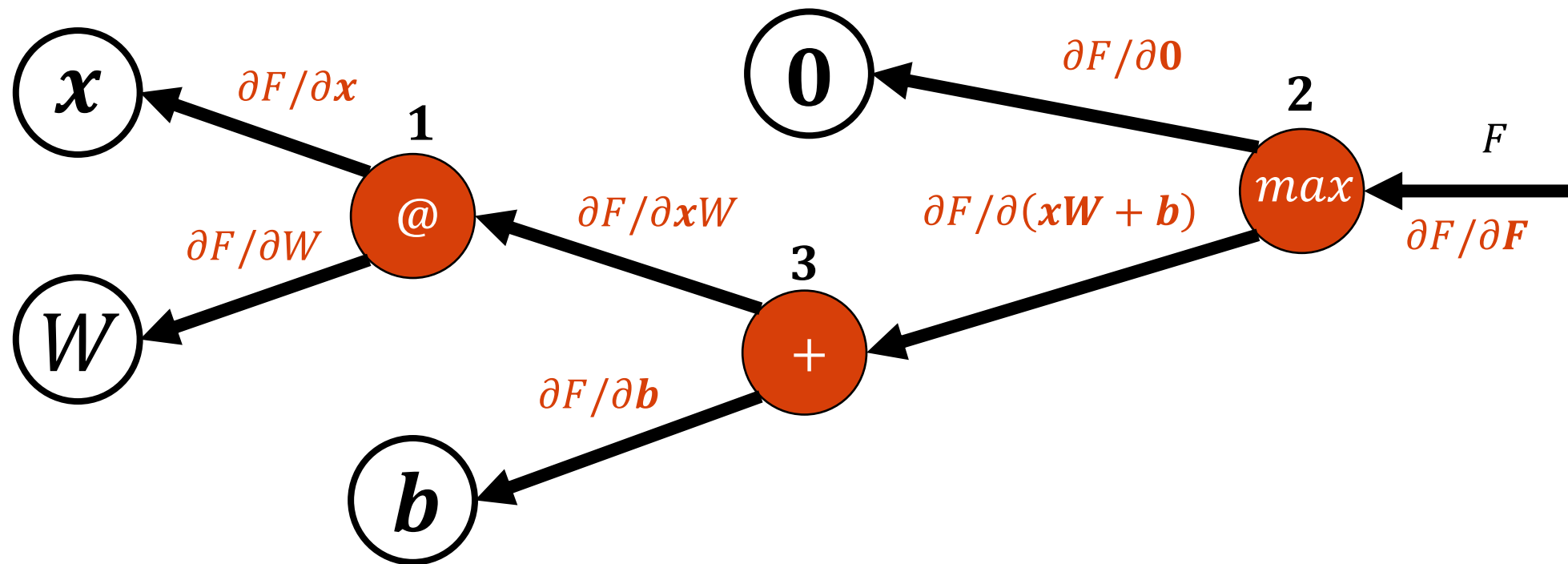
$$\frac{\partial F}{\partial \text{input}} = \frac{\partial F}{\partial \text{output}} \frac{\partial \text{output}}{\partial \text{input}}$$

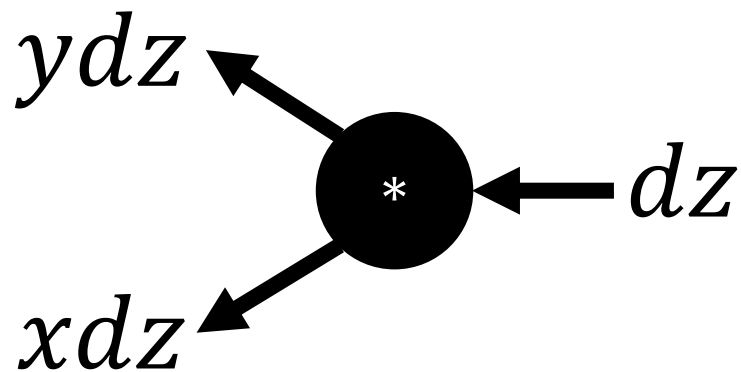
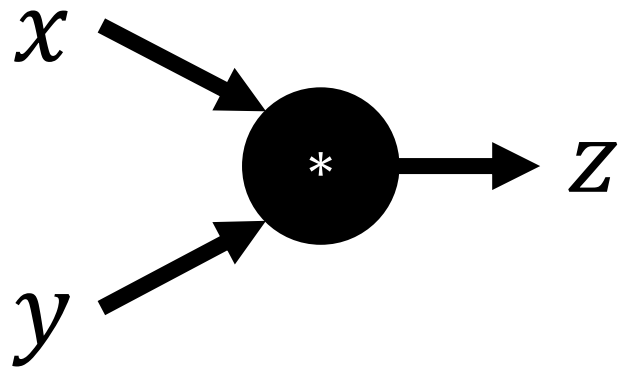




A topological ordering: 2,3,1

$$\frac{\partial F}{\partial \text{input}} = \frac{\partial F}{\partial \text{output}} \frac{\partial \text{output}}{\partial \text{input}}$$





```
class MultiplyGate(object):
```

```
    def forward(x,y):
```

```
        z = x*y
```

```
        self.x = x # must keep these around!
```

```
        self.y = y
```

```
    return z
```

```
    def backward(dz):
```

```
        dx = self.y * dz # [dz/dx * dL/dz]
```

```
        dy = self.x * dz # [dz/dy * dL/dz]
```

```
    return [dx, dy]
```



Implement special variable types (e.g., Tensor in PyTorch) that:

- Override most operations with corresponding forward/backward APIs
- Dynamically build a computation graph as the variables are manipulated

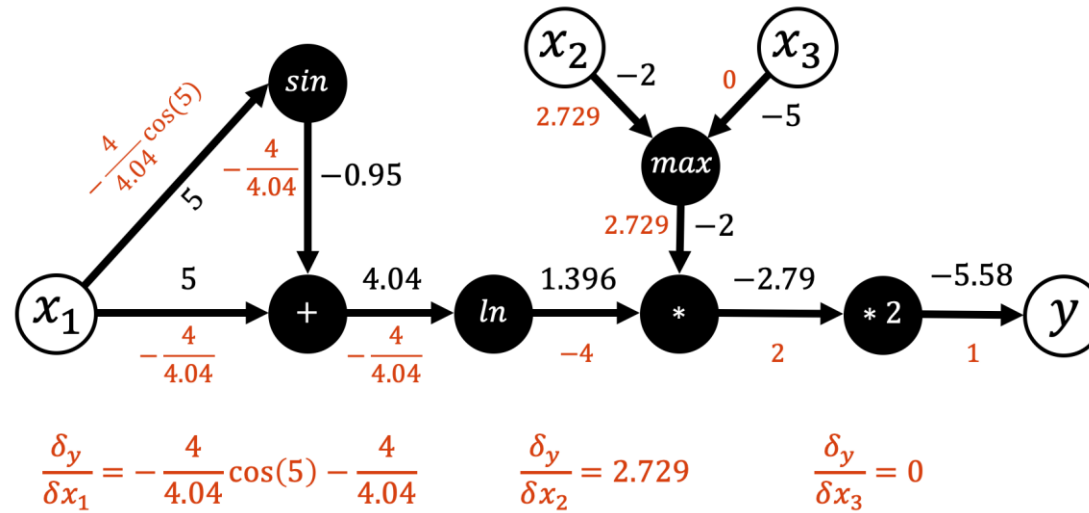
Implement engine that can run backpropagation over the computation graph





Modern Autograd Frameworks -- PyTorch

$$y = 2 \log(x_1 + \sin(x_1)) \max(x_2, x_3)$$

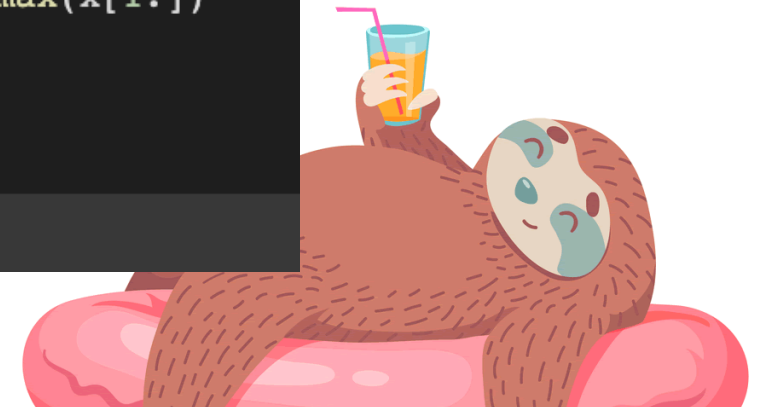


```
import torch
import numpy as np

x = torch.tensor([5.0, -2.0, -5.0], requires_grad=True)
y = 2 * torch.log(x[0] + torch.sin(x[0])) * torch.max(x[1:])

y.backward()
print(x.grad)

tensor([-1.2706,  2.7930,  0.0000])
```





Modern Autograd Frameworks -- PyTorch

```
at::Tensor norm_backward(const at::Tensor & grad, const at::Tensor & self, const optional<at::Scalar> & p_, const at::Tensor & norm);
at::Tensor norm_backward(at::Tensor grad, const at::Tensor & self, const optional<at::Scalar> & p_, at::Tensor norm, at::IntArrayRef dim, bool keepdim);
at::Tensor pow_backward(at::Tensor grad, const at::Tensor & self, const at::Scalar & exponent_);
at::Tensor pow_backward_self(at::Tensor grad, const at::Tensor & self, const at::Tensor & exponent);
at::Tensor pow_backward_exponent(at::Tensor grad, const at::Tensor& self, const at::Tensor& exponent, at::Tensor result);
at::Tensor pow_backward_exponent(at::Tensor grad, const at::Scalar & base, const at::Tensor& exponent, at::Tensor result);
at::Tensor angle_backward(at::Tensor grad, const at::Tensor& self);
at::Tensor mul_tensor_backward(Tensor grad, Tensor other, ScalarType self_st);
at::Tensor div_tensor_self_backward(Tensor grad, Tensor other, ScalarType self_st);
at::Tensor div_tensor_other_backward(Tensor grad, Tensor self, Tensor other);
at::Tensor mvlgamma_backward(at::Tensor grad, const at::Tensor & self, int64_t p);
at::Tensor permute_backwards(const at::Tensor & grad, at::IntArrayRef fwd_dims);
at::Tensor rad2deg_backward(const at::Tensor& grad);
at::Tensor deg2rad_backward(const at::Tensor& grad);
at::Tensor unsqueeze_multiple(const at::Tensor & t, at::IntArrayRef dim, size_t n_dims);
at::Tensor sum_backward(const at::Tensor & grad, at::IntArrayRef sizes, at::IntArrayRef dims, bool keepdim);
at::Tensor nansum_backward(const at::Tensor & grad, const at::Tensor & self, at::IntArrayRef dims, bool keepdim);
std::vector<int64_t> reverse_list(const at::IntArrayRef list);
at::Tensor reverse_dim(const at::Tensor& t, int64_t dim);
at::Tensor prod_safe_zeros_backward(const at::Tensor & grad, const at::Tensor& inp, int64_t dim);
at::Tensor prod_backward(const at::Tensor& grad, const at::Tensor& input, const at::Tensor& result);
at::Tensor prod_backward(at::Tensor grad, const at::Tensor& input, at::Tensor result, int64_t dim, bool keepdim);
at::Tensor solve_backward_self(const at::Tensor & grad, const at::Tensor & self, const at::Tensor & A);
at::Tensor solve_backward_A(const at::Tensor & grad, const at::Tensor & self, const at::Tensor & A, const at::Tensor & solution);
at::Tensor cumsum_backward(const at::Tensor & x, int64_t dim);
at::Tensor logsumexp_backward(at::Tensor grad, const at::Tensor & self, at::Tensor result, at::IntArrayRef dim, bool keepdim);
at::Tensor logcumsumexp_backward(at::Tensor grad, const at::Tensor & self, at::Tensor result, int64_t dim);
```





```
Tensor sum_backward(const Tensor & grad, IntArrayRef sizes, IntArrayRef dims, bool keepdim) {  
    if (!keepdim && sizes.size() > 0) {  
        if (dims.size()==1) {  
            return grad.unsqueeze(dims[0]).expand(sizes);  
        } else {  
            Tensor res = unsqueeze_multiple(grad, dims, sizes.size());  
            return res.expand(sizes);  
        }  
    } else {  
        return grad.expand(sizes);  
    }  
}
```

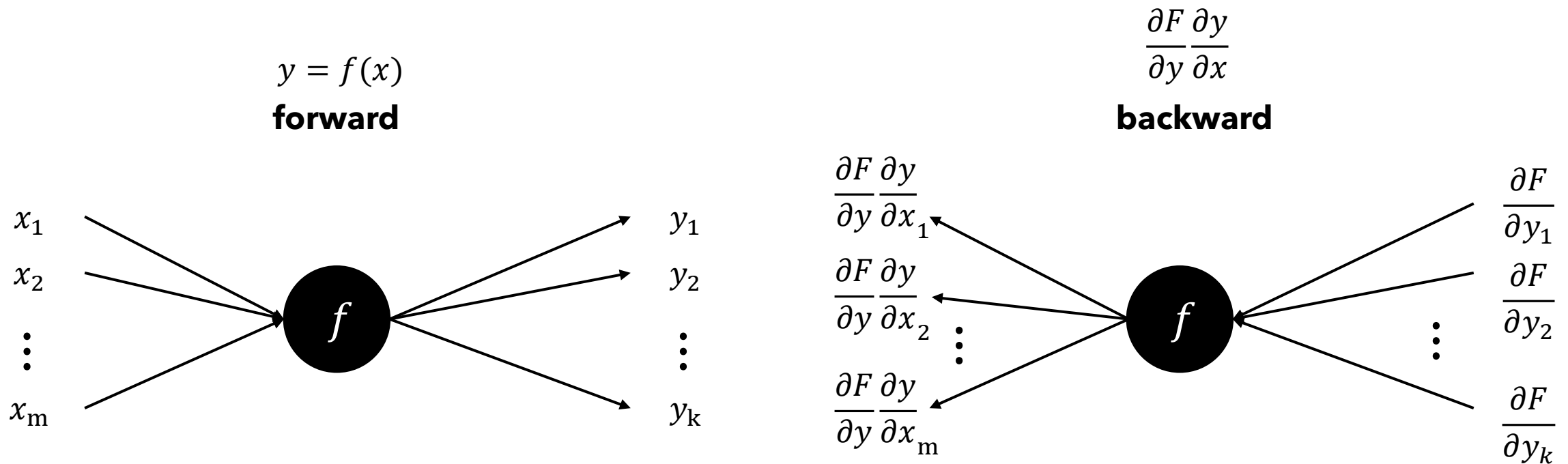




Backpropagation on Computational Graphs

To implement backprop on a graph, each node needs to define:

- Its **forward** computation $y_1, \dots, y_k = f(x_1, \dots, x_m)$
- Its **backward** computation: $\frac{\delta y_i}{\delta x_j} \forall i, j$



Graph representation of computation – nodes are operations, edges values.

- Can be dynamic or statically declared (pros/cons)

Enables forward computation or backpropagation to compute gradients

- Requires a topological sort to ensure all inputs/gradients are there

To implement backprop on a graph, each node needs to define:

- Its **forward** computation $y_1, \dots, y_k = f(x_1, \dots, x_m)$
- Its **backward** computation: $\frac{\delta y_i}{\delta x_j} \forall i, j$

